

Context Selection and Diagnostic Relevance

T. R. Bentley | Context Engineering Paper | September 2026

AI-assisted debugging depends not only on model capability but on which evidence reaches the model, how it is bounded, and whether its provenance remains visible.

This paper defines a typed, failure-directed context-selection architecture for AI Debugger Pro. It ranks evidence by relevance, freshness, trust, privacy risk, injection risk, and token cost.

The attached CSV files and charts are design specifications. They are not measured model-performance or participant results.

Key points

- Repository-grounded baseline: source, error output, bounded project discovery, and Python exception state.
- Core principle: context is a governed evidence package, not a convenient text dump.
- Navigation: Paper 9 -> Paper 10 -> publication catalog.

1. Problem and repository baseline

More context is not automatically better. Large prompts can bury the failing line, expose unrelated secrets, import instruction-like project text, and exhaust the budget before critical evidence appears.

Too little context produces plausible diagnoses untethered from imports, declarations, call sites, configuration, tests, or dynamic state.

The current system provides useful raw material but does not yet expose a unified ranking policy, immutable context manifest, or complete source-to-prompt provenance ledger.

Key points

- Paper 8 measured 87.5% nonempty failure-evidence coverage.
- Paper 9 separates nonempty evidence from actionable evidence.
- Paper 10 governs the package used for diagnosis.

2. Design objectives

The proposed selector optimizes eight objectives without collapsing them into one score: relevance, minimality, freshness, privacy, integrity, reproducibility, language parity, and appropriate abstention.

Key points

- Prefer evidence tied to the failing phase, location, symbol, or call path.
- Exclude material that does not inform a declared diagnostic hypothesis.
- Bind context to source, failure, environment, and proposal hashes.
- Treat repository text as untrusted data, never hidden authority.
- Report insufficiency when required evidence is missing or contradictory.

3. Typed context manifest

Each item carries an identifier, source kind, language, origin, content hash, source-version hash, failure signature, capture time, size estimates, trust class, sensitivity class, selection score, truncation state, and inclusion rationale.

A package becomes immutable when diagnosis begins. Any source or failure change invalidates the package and marks downstream proposals stale.

Key points

- Deterministic ordering supports replication.
- Hashes bind evidence to the reviewed state.
- Rationales make inclusion and exclusion auditable.
- Redaction events are recorded without retaining the secret.

4. Evidence families and pipeline

Direct failure evidence and focal source enter first. Dynamic state, dependency context, behavioral evidence, and project context are admitted through explicit relevance and risk checks.

The pipeline normalizes evidence, generates candidates, filters prohibited material, ranks relevance, allocates a tiered budget, and serializes a reviewable manifest.

Key points

- Critical evidence cannot be displaced by lower-priority files.
- Unused budget may flow downward after reservations are satisfied.
- Missing, stale, or contradictory evidence produces an incomplete-context state.

Failure -> focal source -> dependencies -> tests -> project context

Every transition records relevance, risk, truncation, and provenance.

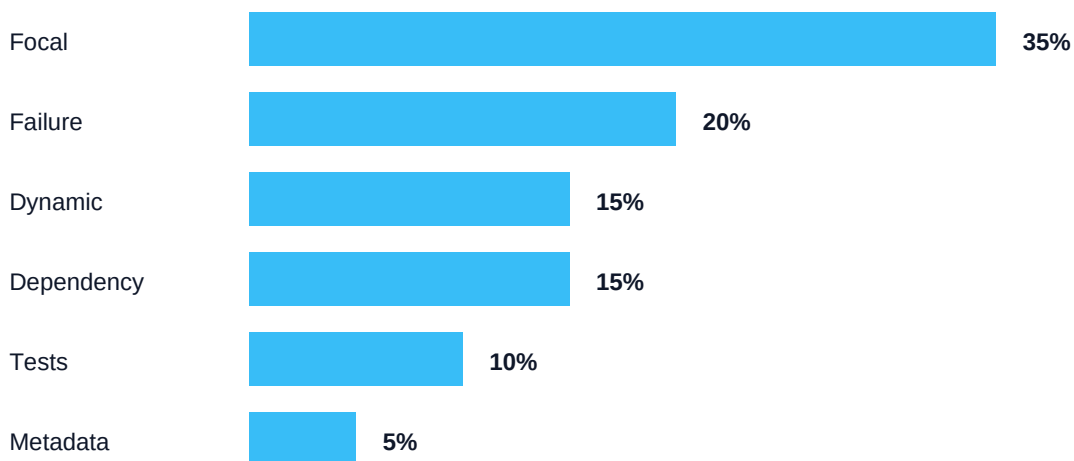
5. Budget and scoring policy

The default design reserves 20% for direct failure evidence, 35% for focal source, 15% for dynamic state, 15% for dependencies, 10% for independent behavioral evidence, and 5% for project metadata.

These values are initial parameters for ablation, not empirically optimized constants. Scores guide selection but are not probabilities of correctness.

Key points

- Positive terms: relevance, proximity, freshness, trust, and expected information gain.
- Negative terms: privacy risk, injection risk, redundancy, and token cost.
- Compare source-only, error-only, unbounded-project, fixed-tier, and adaptive policies.



6. Privacy, security, and prompt injection

Project files, logs, comments, documentation, and generated artifacts may contain instruction-like language. The selector labels origin and trust class and prevents project text from becoming higher-priority instructions.

Secret scanning and path exclusions occur before ranking. Local variables are bounded, redacted, typed, and previewed because they can be simultaneously valuable and sensitive.

Key points

- Never include environment dumps, credentials, caches, binaries, or version-control internals by convenience.
- Hashing low-entropy secrets is not safe anonymization.
- High-utility, high-risk evidence requires redaction or explicit approval.

7. Language-specific evidence

A common manifest supports comparison, while language adapters preserve different semantics. Python prioritizes tracebacks and bounded locals. C and C++ prioritize compiler diagnostics, declarations, headers, return codes, and native runtime evidence. Java adds exception chains, classes, package, classpath, and build metadata.

Key points

- Python: hidden runtime state and dynamic types.
- C: silent exits, macros, and undefined behavior.
- C++: overload ambiguity and diagnostic floods.
- Java: missing dependency and package context.

8. Frozen scenarios and evaluation

Twelve prespecified scenarios cover syntax, compilation, runtime, timeout, missing dependencies, stale evidence, prompt injection, secrets in locals, diagnostic floods, test failures, ambiguous intent, and silent native exits.

Selector-only tests measure deterministic policy behavior. Diagnosis lift requires independently labeled defects and must not be inferred from selection tests.

Key points

- Critical-evidence recall and context precision.
- Sensitive leakage and stale-context rejection.
- Budget compliance and deterministic manifests.
- Blinded root-cause accuracy under context ablations.

9. Human review, roadmap, and conclusion

Before external transmission, the interface should show selected and excluded items, truncation, warnings, estimated size, and reasons. Diagnoses should cite context item identifiers that open the exact local evidence.

Implementation proceeds through manifest foundation, failure-directed selection, governance controls, and preregistered evaluation.

A model cannot reason from evidence it never receives, and it should not receive an entire repository merely because the token window is large. The narrow gate must be typed, bounded, reviewable, reproducible, and honest about insufficiency.

Key points

- Data: context-source-matrix.csv and selection-scenarios.csv.
- Charts: pipeline, proposed budget, and risk-utility map.
- Next planned paper: Regression-Test Generation and Independent Oracles.